

Project Report

Smart Clothes For Smarter You

Billing Management System

Course: Programming in Java (CSE2006)

Evaluation Type: Flipped Course Project Evaluation

Submitted by:

Gopal Kaushik

Registration Number: 25BAI10814

Program: B.Tech Computer Science and Engineering (AI & ML)

Contents

1	Introduction	2
2	The Problem I Wanted to Solve	2
3	What the System Actually Does (Functional Requirements)	2
4	Under the Hood (Non-Functional Requirements)	2
5	System Architecture	3
6	Design Diagrams	3
6.1	Use Case Flow	3
6.2	Process Workflow	3
7	Why I Built It This Way (Design Decisions)	4
8	Implementation Details	4
9	Simulated Terminal Output	4
10	How I Tested It	5
11	The Bugs and Challenges I Faced	5
12	My Key Takeaways	5
13	Future Upgrades	5
14	References	5

1 Introduction

For my flipped course project, I developed the “Smart Clothes For Smarter You” Billing Management System. Instead of just writing a basic Java script, I wanted to build a fully functional, command-line Point of Sale (POS) terminal that brings theoretical Object-Oriented Programming (OOP) concepts to life. The system handles everything from inventory lookups and dynamic shopping carts to automated 18% GST calculation and capturing customer CRM data. By utilizing advanced Java capabilities like the Collections Framework and Multithreading, I aimed to create a checkout experience that feels fast, non-blocking, and fault-tolerant, much like a real-world enterprise application.

2 The Problem I Wanted to Solve

When looking at traditional, manual retail billing or basic software, I noticed a few major bottlenecks that I wanted to fix with my code:

1. **Human Error:** Calculating taxes (like an 18% GST) manually is prone to mistakes, and typos when entering product codes often crash poorly designed applications.
2. **Terminal Freezing (I/O Latency):** A lot of synchronous software freezes while writing receipt data to a hard drive, forcing the cashier to awkwardly wait before serving the next customer.
3. **Slow Searches:** Searching through flat arrays for product codes gets slower and slower (an $O(N)$ time complexity) as the store adds more items.

I built this project specifically to engineer a concurrent, exception-safe solution to these exact problems.

3 What the System Actually Does (Functional Requirements)

I designed the system around three core modules:

- **OOP-Driven Catalog Management:** The system categorizes apparel into Western Wear, Ethnic Wear, and Accessories, displaying them neatly for the user.
- **Dynamic Checkout & Tax Engine:** A workflow that lets the cashier add items to a cart, enter the customer’s name and mobile number, and automatically calculates the exact subtotal and GST.
- **Asynchronous Receipt Generation:** The system takes the final bill and writes it to a `receipts_log.txt` file in the background, without stopping the main terminal loop.

4 Under the Hood (Non-Functional Requirements)

Beyond just making it work, I wanted the code to be efficient and safe. I focused on:

- **Performance:** By using Java’s `HashMap`, I made sure that looking up a product code happens instantly in $O(1)$ constant time, no matter how big the catalog gets.
- **Reliability:** I hate when applications crash from a simple typo, so I built robust `try-catch` blocks and a custom `InvalidProductCodeException` to handle bad inputs gracefully.

- **Concurrency / Data Safety:** I offloaded the file writing to a background `Thread` to keep the terminal responsive.
- **Maintainability:** The code is strictly modular. I used OOP principles to separate the data models, the business logic, and the file I/O operations into different classes.

5 System Architecture

I split the application into a clean 3-layer architecture:

1. **User Interface Layer:** The terminal loop (`BillingSystemMain`) that takes commands and inputs from the cashier.
2. **Business Logic Layer:** The `StoreManager` which validates codes, does the math, and handles the shopping cart data.
3. **Data Persistence Layer:** The `ReceiptGeneratorThread`, a dedicated background worker that formats and saves the invoice to the local disk.

6 Design Diagrams

6.1 Use Case Flow

- **Actor:** Cashier
- **Actions:** View Catalog → Add Item to Cart → (Handle Invalid Inputs if any) → Enter Customer Details → Checkout.
- **System Action:** Spawn Background Thread → Write to `receipts_log.txt`.

6.2 Process Workflow

1. System boots and loads the Catalog into memory.
2. Cashier types a Product Code.
3. If the code is invalid, the system throws an `InvalidProductCodeException` and asks again.
4. If valid, the system asks for quantity, validates it as an integer, and adds it to the `ArrayList` cart.
5. Cashier types `BILL` to checkout.
6. System prompts for Customer Name and Mobile.
7. System calculates totals, passes a deep-copy of the cart to the `ReceiptGeneratorThread`, and immediately resets for the next customer.

7 Why I Built It This Way (Design Decisions)

- **Why a `HashMap` over an `ArrayList` for the inventory?** Searching an array means checking every single item until you find a match. Using a `HashMap` lets the system jump straight to the product using its code as a key, making it exponentially faster.
- **Why use `Polymorphism`?** I created an abstract `Product` class so that if the store decides to sell Shoes next month, I can just add a new child class without rewriting the core billing engine.
- **Why `Multithreading`?** Standard `FileWriter` operations block the program while saving to a disk. By pushing the receipt writing to a secondary thread, the cashier isn't forced to wait for the hard drive to finish writing.

8 Implementation Details

To keep things organized and meet the syllabus requirements, I structured the project using 8 distinct classes:

1. `Product.java` (The Abstract Base)
2. `WesternWear.java`, `EthnicWear.java`, `Accessory.java` (The Child Classes)
3. `CartItem.java` (To bundle products with quantities)
4. `InvalidProductCodeException.java` (For my custom error handling)
5. `ReceiptGeneratorThread.java` (To handle the asynchronous I/O)
6. `StoreManager.java` (The core brain handling the `HashMaps` and logic)
7. `BillingSystemMain.java` (The main loop where the program starts)

9 Simulated Terminal Output

```
=====
      Smart Clothes For Smarter You
      BILLING & CHECKOUT SYSTEM
=====

--- PRODUCT CATALOG ---
Code: ACC01 | [Accessory] Leather Belt      | Price: $12.50
Code: ETH01 | [Ethnic]   Silk Kurta        | Price: $35.00
Code: WST01 | [Western]  Denim Jacket       | Price: $45.99
-----
Enter Product Code: WST01
Enter Quantity: 2
-> Added 2x Denim Jacket to cart.

Enter Product Code: BILL
--- CHECKOUT PROCESS ---
Enter Customer Name: Gopal Kaushik
Enter Customer Mobile Number: 9876543210
```

Checkout complete for Gopal Kaushik! Grand Total: \$108.54

[SYSTEM] Receipt successfully saved to receipts_log.txt for Gopal Kaushik

10 How I Tested It

- **Trying to break the inputs:** I intentionally typed garbage values like “XYZ123” for product codes and “five” for quantities. My custom exceptions and `NumberFormatException` catch blocks successfully prevented the program from crashing.
- **Math Verification:** I ran several multi-item carts through the system and manually calculated the 18% GST to ensure the floating-point math was perfectly accurate.

11 The Bugs and Challenges I Faced

- **The Scanner Buffer Bug:** When I mixed `scanner.nextInt()` with `scanner.nextLine()`, the terminal kept skipping the prompt for the customer’s name because of a leftover new-line character in the buffer. I fixed this by uniformly reading everything as a string using `scanner.nextLine()` and parsing the integers manually.
- **Thread Race Conditions:** At first, my background thread kept printing empty receipts. I realized that the main thread was clearing the shopping cart array faster than the background thread could print it! I fixed this by creating a “deep copy” of the cart (`new ArrayList<>(cart)`) and passing that specific copy to the thread before clearing the main cart.

12 My Key Takeaways

This project was a huge step up from writing basic scripts. It forced me to actually apply theoretical concepts like Encapsulation and Polymorphism to solve real problems. I also gained a much deeper understanding of how Java handles memory via the Collections Framework, and how tricky (but powerful) concurrent programming with Multithreading can be.

13 Future Upgrades

If I were to expand this project later, I would:

- Connect it to a real MySQL database using JDBC instead of hardcoding the `HashMap` catalog.
- Build a Graphical User Interface (GUI) using JavaFX to make it look like a modern touch-screen POS.

14 References

- Schildt, H. (2018). *Java: The Complete Reference*, 11th Edition. Oracle Press.
- Course Materials: CSE2006 — Programming in Java Syllabus & Modules.
- Oracle Official Java Documentation: Collections Framework and Concurrency APIs.